

category: computer-vision

tags: [opencv, otsu, adaptive-threshold, license-plate, contour, image-segmentation]

severity: high

created: 2026-07-28

updated: 2026-07-28

project-origin: TraningTiengLao

Otsu threshold trên biển số Lào thất bại khi ảnh có viền/nền trắng quanh biển (không phải crop sát)

Tình huống gặp phải

Auto-label ký tự dòng 2 biển số Lào để tạo YOLO label. Script `auto_label_lao_plates.py` được viết/test ổn với 2 ảnh mẫu đã crop sát biển số (crop nghiêng, Roboflow augmentation). Khi chạy tiếp trên ảnh trong `label/train/` (ảnh full biển số, còn nền trắng + viền đen quanh, ốc vít, dòng 1 phía trên) thì toàn bộ ảnh cho ra 0 box.

Triệu chứng / Lỗi

```
[PROCESSED] train/xxx.jpg -> xxx.txt (0 labels)
```

Toàn bộ 5 ảnh test đều 0 box. Debug `extract_line2_boxes`:

- `cv2.threshold(..., THRESH_BINARY_INV + THRESH_OTSU)` trên vùng dòng 2 (crop theo % chiều cao ẢNH GỐC) ra kết quả gần như toàn ảnh trắng (1 contour bao trọn cả vùng) hoặc ngược lại vài chấm nhiễu nhỏ — không tách được ký tự.
- Thử `adaptiveThreshold` (blockSize 35) thì bị lỗi khác: ký tự ra dạng RỖNG (chỉ viền ngoài, không fill), contour bị vỡ thành nhiều mảnh nhỏ rời rạc → mất hẳn 2/3 ký tự số giống nhau liền kề ("666").

Nguyên nhân gốc rễ (Root Cause)

Otsu là ngưỡng toàn cục dựa trên histogram 2 mode (nền/chữ). Ảnh "full biển số" có thêm vùng nền trắng lớn ngoài viền đen của biển (do bounding-box crop từ object detection rộng hơn biển thật) → histogram grayscale thành đa mode (nền trắng, viền đen, nền vàng biển, chữ đen) → Otsu chọn ngưỡng sai, không tách nổi chữ khỏi nền vàng. `adaptiveThreshold` với blockSize quá nhỏ so với độ dày nét chữ (~130px cao) lại gây hiệu ứng "chỉ bắt viền" (giá trị pixel giữa nét đậm gần bằng trung bình cục bộ nên không vượt ngưỡng).

Giải pháp

- Tìm vùng biển số thật trước bằng color mask HSV (nền biển Lào luôn vàng): `cv2.inRange(hsv, [15, 60, 80], [45, 255, 255])`, lấy contour lớn nhất, bounding box, inset ~3% để bỏ viền đen.
- Tính vùng "dòng 2" theo % chiều cao của BOUNDING BOX BIỂN SỐ (không phải ảnh gốc):
`line2_y1=0.42*ph, line2_y2=0.98*ph.`

3. Trong vùng biển đã crop sạch (không còn nền trắng ngoài), Otsu global lại hoạt động tốt — không cần đổi sang adaptiveThreshold.
4. Convert toạ độ box từ hệ toạ độ "trong vùng biển" về hệ toạ độ ảnh gốc bằng cộng offset (`px`, `py`).
5. Filter contour theo `bw < pw*0.85` (loại bỏ contour viền ngoài lẫn vào) thay vì so theo chiều rộng ảnh gốc.

Áp dụng lại (How to reuse)

- Khi thấy pipeline threshold hoạt động tốt trên ảnh crop sát nhưng ra 0 box/box sai trên ảnh có thêm nền/viền quanh chủ thể → nghi ngay Otsu bị nhiễu bởi vùng nền ngoài, không phải do logic contour/merge.
- Luôn tách bước "định vị chủ thể chính" (ở đây: biển số màu vàng) trước bước "tách ký tự trong chủ thể" — 2 bước threshold khác mục đích, không dùng chung 1 ngưỡng cho cả 2.
- Debug bằng cách lưu ảnh threshold ra file và Read lại để xem trực quan (không chỉ đếm contour) — nhìn ra ngay lỗi "toàn trắng" hay "chỉ viền rỗng" nhanh hơn đoán bằng số liệu.

Chú ý / Cạm bẫy (Gotchas)

- `adaptiveThreshold` không phải lúc nào cũng "an toàn hơn" Otsu — blockSize phải tương ứng độ dày nét chữ, sai blockSize gây rỗng ruột ký tự, tệ hơn Otsu.
- `color mask` không tìm được biển (ví dụ ảnh chụp thiếu sáng/ổ màu) — luôn có fallback trả (`0,0,w,h`) (dùng cả ảnh) để không crash, nhưng cần log lại để biết ảnh nào rơi vào fallback (chất lượng label sẽ kém hơn).
- Ký tự số giống nhau liền kề (VD "666") dễ bị contour-merge dính thành 1 box nếu blockSize/threshold sai — luôn test trên biển có ký tự trùng lặp liên tiếp, không chỉ test biển có ký tự đa dạng.

Phụ lục 1 — Contour đường kẻ ngang làm bước gộp-chồng-lấn NUỐT HẾT ký tự

Triệu chứng: ảnh biển rõ nét, mắt đọc được 6 ký tự, nhưng detector trả về đúng 2 box

(hoặc 1) — không exception, không cảnh báo.

Nguyên nhân: viền dưới biển / bóng đổ / vệt bẩn kéo dài sinh ra 1 contour **rất rộng và thấp**

(VD `x=0`, `y=141`, `w=342`, `h=24`). Contour này có `w,h >= 10` nên lọt qua filter, và vì `x=0` nên sau

`sort(key=x)` nó nằm **đầu danh sách**. Bước gộp box theo độ chồng lấn ngang duyệt tuần tự:

box kẻ ngang chồng với ký tự 1 → gộp → box gộp còn rộng hơn → chồng với ký tự 2 → gộp → ...

đổ domino nuốt gần hết ký tự thành 1 box khổng lồ.

Fix: loại contour có tỉ lệ bất thường **TRƯỚC** khi gộp, không phải sau:

```
if bw > bh * 2.0 or bh < line2_h * 0.35:
    continue # đường kẻ ngang / vệt quá thấp, không thể là ký tự
```

(ký tự Lào luôn cao hơn rộng, và cao $\geq \sim 35\%$ chiều cao dải dòng 2)

Áp dụng lại: để thấy số box trả về **ít bất thường** (1-2 box trên ảnh rõ nhiều ký tự) →

in ra `raw_boxes` **TRƯỚC** bước gộp. Nếu raw đúng mà sau gộp sai thì lỗi ở merge, không phải threshold.

Mọi thuật toán gộp tuần tự theo chồng lấn đều có rủi ro domino này — phải lọc nhiều dạng

"1 phần tử rất lớn" trước khi vào vòng gộp.

Phụ lục 1b — Hard-code số lượng chữ số làm ký tự CHỮ cuối bị ép sang nhóm SỐ

Biển Lào dòng 2 có nhiều độ dài: **๒๓ 6399** (2 chữ + 4 số) nhưng cũng có **๐๖7** (2 chữ + 3 số).

Code ràng buộc vị trí kiểu `n_letters = n_boxes - 4` (hard-code 4 số) thì với biển 5 ký tự sẽ tính ra 1 chữ + 4 số → ký tự chữ thứ 2 (๓) bị ép chọn trong nhóm chữ số → nhầm sai.

Dấu hiệu nhận biết: đúng 1 ký tự có confidence tụt thẳm (0,13) trong khi các ký tự khác $\geq 0,9$.

Đừng vội nghĩ ảnh mờ hay thiếu mẫu train — kiểm tra ràng buộc vị trí trước.

Fix: không cố định số lượng, mà DÒ điểm chia `k` tối ưu (vị trí $< k$ là chữ, $\geq k$ là số), chọn `k` làm tổng log-xác suất lớn nhất:

```
logp = np.log(prob + 1e-12)
best_letter, best_digit = logp[:, ~is_digit].max(1), logp[:, is_digit].max(1)
k = max(range(1, n), key=lambda k: best_letter[:k].sum() + best_digit[k:].sum())
```

Sau fix, ký tự đó lên **0,96**. Cách này tự xử lý mọi biến thể 2+3, 2+4, 3+4... mà vẫn giữ được lợi ích của ràng buộc (loại hẳn nhóm nhầm chữ↔số).

Phụ lục 1c — 3 lỗi nữa cùng họ "ngưỡng cứng" (đo trên 200 ảnh ngẫu nhiên)

(a) Cắt dải dòng chữ theo tỉ lệ cố định làm CỤT ĐỈNH mọi ký tự.

`line2 = plate[0.42*ph : 0.98*ph]` — tỉ lệ dòng1/dòng2 khác nhau theo từng biển, nên dải cắt bắt đầu BÊN DƯỚI đỉnh glyph. Dấu hiệu chẩn đoán rất rõ: **mọi raw box đều có `y == 0`**.

Hậu quả kép: bbox xuất ra thiếu phần trên, và crop đưa vào classifier bị mất đầu nên đọc sai.

Fix: bỏ cắt theo tỉ lệ, dò contour trên TOÀN vùng biển rồi chọn "hàng chữ cao"

`(bh >= 0.55 * hmax + cùng dải tâm y)` — dòng 2 luôn cao hơn hẳn tên tỉnh ở dòng 1.

(b) Ngưỡng pixel tuyệt đối chết trên ảnh nhỏ. Ảnh nguồn chệnh ~30 lần (nhỏ nhất 22×11, trung vị 640×380; ~9% cao dưới 100px). `bw>=10, bh>40` loại sạch ký tự của ảnh nhỏ → trả 0 box im lặng. Fix: co/giãn vùng biển về chiều cao chuẩn (`PLATE_H=220`) rồi đặt MỌI ngưỡng theo tỉ lệ của không gian chuẩn hoá đó, cuối cùng chia `scale` để trả toạ độ về ảnh gốc.

(c) Mở rộng mask màu để "bắt được nhiều hơn" lại làm TỆ ĐI — phải đo, đừng suy luận.

Biển Lào có cả nền trắng/cam/olive bạc màu, nên trực giác là gộp thêm mask trắng

`(s<70, v>120)` vào mask vàng. Kết quả **ngược lại**: số ảnh tách được 5–6 ký tự tụt **140 → 82**,

vì vùng sáng của nền ảnh (trời, thân xe, mặt đường) cũng khớp và kéo bbox ra ngoài tầm biển.

Fix đúng: mask vàng là **chính**, mask trắng chỉ chạy **dự phòng** khi mask vàng cho bbox không

hợp lệ ($< 30\%$ ảnh) → 0-box giảm 37 → 30, 5–6 ký tự tăng 140 → 145.

⚠ Bài học chung của cả (a)(b)(c): mọi hằng số "có vẻ hợp lý" trong pipeline CV cổ điển đều là một giả định ngầm về dữ liệu. Với dataset ảnh thực tế đa dạng, **mỗi bản vá heuristic đều có khả năng gây hời quy ở chỗ khác** — phải có script đo trên mẫu ≥ 200 ảnh và so số trước/sau cho TỪNG thay đổi riêng lẻ (ablation), không gộp nhiều thay đổi rồi đoán cái nào hiệu quả.

Trạng thái sau tất cả các fix (200 ảnh ngẫu nhiên): 145 ảnh (73%) ra 5–6 ký tự;

30 ảnh (15%) ra 0 ký tự; **68 ảnh (34%) "sạch"** (5–6 ký tự + mọi confidence $\geq 0,7$);

23% ký tự bị gán cờ. Đây là trần thực tế của pipeline contour + CNN phân loại rời.

Phụ lục 2 — Hand-crafted features KHÔNG thắng được template matching ở bài này; CNN thì có

Đã thử nghiệm túc hướng trích xuất đặc trưng tường minh (hướng vòng cung, độ đậm cạnh, khắc đỉnh, lề trái theo dải, vòng khép kín, gấp góc...) cho 20 class chữ Lào + 10 số, ~165 mẫu.

Đo bằng leave-one-out kNN + ablation theo nhóm đặc trưng ([ablation.py](#)):

Phương án	Chữ cái (20 class)	Số (10 class)
---	::	::
Chỉ zoning (mật độ mực lưới 4×3 — template matching thô)	67,3%	82,7%
Chỉ đặc trưng cấu trúc tự thiết kế (arc/apex/margin/stroke/corner/holes)	13–40% mỗi nhóm	19–73%
Tổ hợp tốt nhất (greedy forward selection)	77,0%	86,5%
CNN nhỏ 2 kênh (xám CLAHE + nhị phân), 32×64, augment affine/morph/blur	96,0% (5-fold, std 3,3)	—

⚠ **Kết luận trái với trực giác ban đầu:** **zoning** — vốn chính là template matching thô — lại là nhóm đặc trưng **mạnh nhất**, còn các đặc trưng cấu trúc "thông minh" tự thiết kế yếu hơn nhiều khi dùng riêng. Lý do: trên ảnh biến thật (mờ, nghiêng, tương phản kém) mọi đặc trưng cục bộ đều nhiễu nặng, trong khi mật độ mực theo vùng lại ổn định.

Bài học: khi bộ ký tự khác nhau ở chi tiết nhỏ và chỉ có ~5 mẫu/class, **đừng đầu tư vào hand-crafted features** — trần thực tế chỉ ~77–86%. Với torch có sẵn, CNN nhỏ + augment mạnh đạt 96% trong vài phút. Hand-crafted features vẫn có giá trị ở chỗ khác: dò cấu trúc thô (hướng vòng cung) đạt **100% nhất quán** và dùng tốt để **phát hiện mẫu lỗi / gán nhãn sai** trong dataset ([eval_features.py](#) tìm ra 6 mẫu hỏng: nhị phân hoá thất bại, crop lẫn ký tự bên cạnh, nghi sai nhãn).

Mẹo kèm theo: nhị phân hoá phải thử **nhiều kênh** (gray, CLAHE(gray), a, b của LAB, saturation) × 2 chiều tương phản rồi **chấm điểm chọn phương án tốt nhất** (tỉ lệ mực hợp lý, 1 thành phần liên thông chiếm ưu thế, không chạm cả 4 mép). Otsu trên grayscale đơn thuần thất bại với chữ vàng sáng trên nền trắng và nâu trên vàng nâu — kênh a/b cứu được vì tách theo SẮC ĐỘ chứ không theo độ sáng.

Tham chiếu

- Project liên quan: TraningTiangLao — [auto_label_lao_plates.py](#) ([find_plate_bbox](#), [extract_line2_boxes](#)), [lao_features.py](#), [eval_features.py](#), [ablation.py](#), [train_cnn.py](#), [predict.py](#)

- Lesson liên quan: [lao-plate-read-lao-letters-loop-position.md](../camera-integration/lao-plate-read-lao-letters-loop-position.md)